Microsoft AI Engineer Program via Simplilearn

Submitted by Ronald M. Paraison

Lending Club Loan Data Analysis

Deep Learning Specialization — Course-end Project Writeup

Problem

LendingClub needs to know upfront whether a loan is likely to go bad, meaning it won't get fully repaid (`not.fully.paid`). Getting this right lets them price loans correctly and avoid funding the ones headed for default. The data covers loans from 2007 to 2015.

Data

`loan_data.csv` : 9,578 loans, 14 columns. `purpose` is the only categorical field. Everything else is numeric: interest rate, installment, income, DTI, FICO, credit line age, revolving balance and utilization, recent inquiries, delinquencies, public records. Target is `not.fully.paid` , 0 or 1.

What I did

EDA. No missing values anywhere. The target is skewed: 83.99% fully paid vs. 16.01% default. A model that just predicts "fully paid" every time would already hit ~84% accuracy while being completely useless, which is why accuracy isn't the number to chase on this one. Plotted the numeric features and checked FICO against default status. Borrowers who default skew toward lower FICO and higher rates, which is exactly how lending is supposed to work, so that was a good sanity check on the data itself.

Feature transformation. `purpose` has 7 values (`debt_consolidation` , `credit_card` , `all_other` , `home_improvement` , `small_business` , `major_purchase` , `educational`) with no real

order to them, so it got one-hot encoded rather than label-encoded. That took the feature count from 13 to 18.

Feature engineering. Ran a correlation matrix across all 18 features and checked for pairs above 0.75. Nothing crossed that line. `int.rate` and `fico` move together in the direction you'd expect (riskier borrower, higher rate) but not enough to call either one redundant, so all 18 features stayed.

Modeling. Scaled everything with `StandardScaler`, split 80/20 stratified so both sets keep the same class balance. Built a small feed-forward network in Keras: Dense(64, relu) → BatchNorm → Dropout(0.3) → Dense(32, relu) → BatchNorm → Dropout(0.2) → Dense(16, relu) → Dense(1, sigmoid). Adam optimizer, binary cross-entropy, tracked accuracy/AUC/recall. Because of the imbalance I computed class weights (0: 0.60, 1: 3.12) and applied them during training — without that, the network just learns to predict "fully paid" for everything and calls it a day. Early stopping on validation AUC cut training off at epoch 28.

Results

Metric	Value
Class balance (fully paid / default)	83.99% / 16.01%
Features after encoding	18
Feature pairs dropped (correlation > 0.75)	0
Training epochs (early-stopped)	28
Test accuracy	0.660
Test recall (default class)	0.583
Test ROC-AUC	0.694

Notes

ROC-AUC (0.694) matters more here than accuracy (0.660), since accuracy is inflated by the imbalance. The class weighting is a deliberate tradeoff. It costs precision to gain recall on the default class, and for a lender that's usually the right call, since missing a real default is worse than flagging a safe borrower who turns out fine. This covers what the assignment asked for: transform the categorical field, run EDA, check correlations, build a Keras model that produces a usable default-risk score.